TSFM-Lens: Controlled, Causal and Confirmatory Interpretability of Time-Series Foundation Models

Daniel Kushnir KUSHNIR.DANIEL05@GMAIL.COM
Hao Wang (corresponding author) HOGUEWANG@GMAIL.COM
Rutgers University

Abstract

TSFM-Lens is an open-source Python toolkit that turns a list of time-series foundation model (TSFM) checkpoints into a single self-contained interpretability report with one command. It compares models with different architectures on a measured common time axis, validates every probe on a synthetic benchmark with exact generative ground truth, defines concepts by their causal effect on the forecast against matched random nulls, and tests exploratory claims once on a sealed held-out split. Every reported number carries an evidence class and a plain-language explanation. The toolkit ships adapters for TimesFM, the Chronos family, Sundial, Timer, Time-MoE and Lag-Llama, a zero-code adapter for Hugging Face checkpoints, a complete seven-model example run with held-out confirmation and over two thousand tests. It is released under the MIT license at `https://github.com/DanK10097110/TSFM-Interp` (archived as doi:10.5281/zenodo.23267463).

Keywords: interpretability, time-series foundation models, sparse autoencoders, causal intervention, open-source software

1 Introduction

Pretrained forecasters such as TimesFM (Das et al., 2024) and Chronos (Ansari et al., 2024) are used zero-shot, yet little is known about what they compute. Interpretability libraries built for language models work on one model at a time, and three properties of TSFMs make a direct transfer unreliable: tokens cover 1 to 96 time steps depending on the architecture, so positions are not comparable across models; real series carry no labels for trend, seasonality or noise, so probes cannot be checked; and the causal effects of internal features are small enough that an uncontrolled measurement yields a confident false positive. TSFM-Lens is a comparative tool built around these problems: its unit of analysis is a panel of models on a shared benchmark, and its report is written for newcomers and auditable by experts.

2 Architecture

The repository contains two separately installable packages (Figure 1). `tsfm_benchmark` builds and validates corpora; `tsfm_lens` analyzes models and renders the report; the benchmark can be used on its own.

Benchmark. Generators come in two labeled tiers: a synthetic tier that touches no real data and records every generative component, and a real-derived tier with attributional ground truth. Samples pass a leakage audit against real archives; dev and private splits use disjoint seeds, are hashed and can be re-minted as fresh epochs. Structured corruptions and single-knob counterfactuals give perturbations with known effects.

©2026 Daniel Kushnir and Hao Wang.

License: CC-BY 4.0, see `https://creativecommons.org/licenses/by/4.0/`.

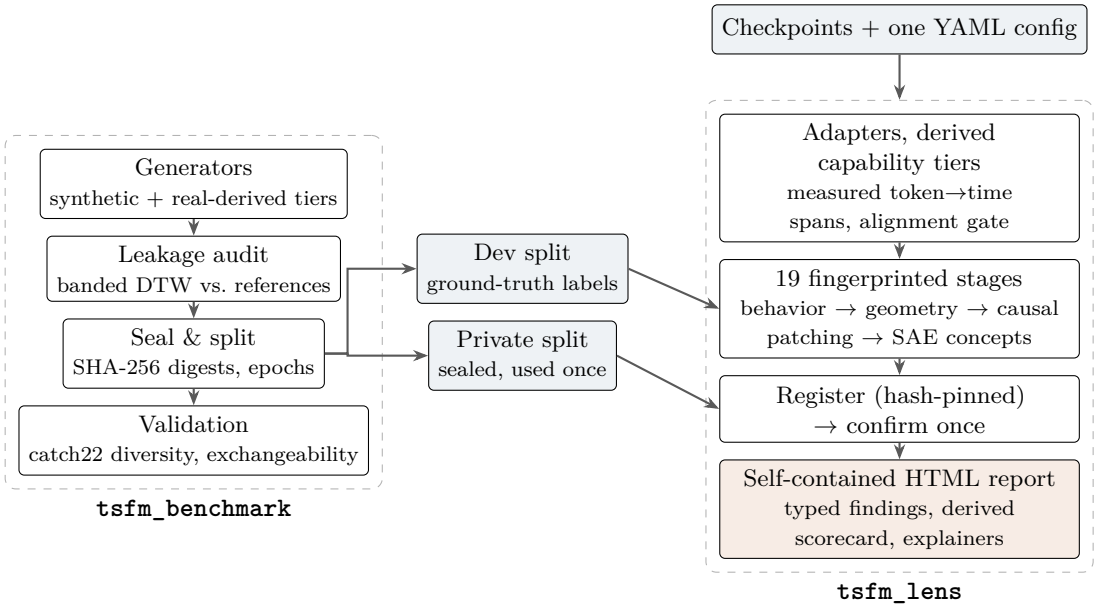


Figure 1: Architecture of TSFM-Lens. The benchmark package produces leakage-audited, sealed corpora with exact ground truth. The analysis package loads each checkpoint through an adapter, runs a gated stage pipeline on the dev split, confirms registered claims once on the private split and renders one report.

Adapters and alignment. An adapter exposes loading, prediction, the module and token time spans; its capability tier is derived from the methods it implements, and stages needing a higher tier are skipped with the reason rendered. A zero-code adapter measures token spans from impulse responses for Hugging Face checkpoints and refuses rather than guesses. Tokens are overlap-pooled onto common time windows, verified per layer by an alignment gate, so every cross-model statistic compares the same stretch of time.

Stages. Nineteen stages produce evidence of increasing strength: behavioral (accuracy, calibration, cost), descriptive (depth profiles, lenses), geometric (CKA, stitching gain over an input baseline), causal within-model (corruption fingerprints, patching, head ablation) and concept-level. A sparse-autoencoder feature counts as causal only if removing it changes a forecast channel more than removing the same per-token amount along a random direction on the same series, after reach controls confirm the intervention lands; a shipped planted-concept forecaster holds this test to a fixed false-positive gate; concepts are then matched across models on two levels, same inputs and same effect on those inputs. Stage inputs are fingerprinted, so cached artifacts are reused only while valid.

Statistics and report. Intervals resample series, not windows; a multiplicity ledger flags correction families that cannot reach significance; and exploratory claims are frozen in a hash-pinned registry and tested once on the sealed split. The output is one self-contained HTML file whose sections pose plain questions, whose figures each carry an explanation of how to read them, and whose every claim states its evidence class and a generated caveat.

Capability	*TransformerLens*	*nnsight / pyvene*	*SAELens*	*Captum*	*TSFM SAE studies*	**TSFM-Lens**
Hooks, caching, activation patching	✓	✓	○	○	○	✓
Sparse dictionaries and feature ablation	○	○	✓	✗	✓	✓
TSFMs without per-model engineering	✗	○	✗	○	✗	✓
Measured token→time alignment across architectures	✗	✗	✗	✗	✗	✓
Benchmark with exact ground truth and leakage audit	✗	✗	✗	✗	○	✓
Profile-matched random-direction nulls and reach controls	✗	✗	✗	✗	✗	✓
Known-answer forecaster with planted concepts and decoys	✗	✗	✗	✗	✗	✓
Cross-model concept matching (inputs vs. effects)	✗	✗	✗	✗	✗	✓
Series-level resampling and multiplicity ledger	✗	✗	✗	✗	✗	✓
Registered one-shot confirmation on sealed data	✗	✗	✗	✗	✗	✓
One-command report with typed evidence per claim	✗	✗	○	✗	✗	✓

Table 1: Capabilities for comparative TSFM interpretability: ✓ provided, ○ partial or possible with user code, ✗ not provided. Libraries: Nanda and Bloom (2022); Fiotto-Kaufman et al. (2025); Wu et al. (2024); Bloom et al. (2024); Kokhlikyan et al. (2020); TSFM SAE studies judged from the papers.

3 Comparison with Existing Tools

General interpretability libraries are single-model microscopes: they supply hooks, caching, patching and sparse dictionaries, and leave the experimental design to the user. Recent TSFM studies train sparse autoencoders on one model (Mishra, 2026; Oublal et al., 2026; Divo et al., 2026). TSFM-Lens builds on the same primitives but adds what a cross-architecture comparison needs to be trustworthy (Table 1): a measured time axis, known answers, matched nulls with reach controls, correction for multiple testing, one-shot confirmation and a report that states the evidence behind every claim.

4 Usage, Quality and Availability

The packages install from PyPI (`pip install tsfm-lens tsfm-benchmark`). From a clone, after `pip install -e . && pip install -e tsfm_lens`, one command runs a full analysis (`python run.py --config configs/examples/large_4model.yaml`); presets cover a CPU-only mock smoke test, a single model and a pair. The repository includes a complete seven-model example run (`examples/panel7_v2`) with its report, per-stage artifacts and one-shot held-out confirmation (including 20 of 20 registered concept-transfer claims confirmed on 971 sealed series, and again on a second, three times larger corpus and on 800 raw real windows), and a ledger of every finding with its reproduction command. The code is MIT-licensed; over 2,600 tests run in continuous integration against pinned and, weekly, latest dependencies, and each load-bearing assertion must be shown to fail against a planted regression. Code: `https://github.com/DanK10097110/TSFM-Interp`.

Acknowledgments and Disclosure of Funding

Computational resources were provided by the second author's lab at Rutgers University. This work received no external funding, and the authors declare no competing interests. Use of AI tools: Claude (Anthropic) wrote most of the repository's code and tests, ran the experiments and helped draft and edit the manuscript, under the authors' direction. The research questions, ideas and general design choices are the authors', who reviewed the implementation details Claude proposed, checked every reported result and take full responsibility for the content.

References

A. F. Ansari, L. Stella, C. Turkmen, X. Zhang, P. Mercado, H. Shen, O. Shchur, S. S. Rangapuram, S. Pineda Arango, S. Kapoor, J. Zschiegner, D. C. Maddix, H. Wang, M. W. Mahoney, K. Torkkola, A. G. Wilson, M. Bohlke-Schneider, and Y. Wang. Chronos: Learning the language of time series. *Transactions on Machine Learning Research*, 2024.

J. Bloom, C. Tigges, A. Duong, and D. Chanin. SAELens. `https://github.com/jbloomAus/SAELens`, 2024.

A. Das, W. Kong, R. Sen, and Y. Zhou. A decoder-only foundation model for time-series forecasting. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, 2024.

F. Divo, M. Kraus, R. Härle, P. Ahrend, and K. Kersting. Finding the Zeitgeist in time series foundation models. In *ICLR 2026 Workshop on Time Series in the Age of Large Models*, 2026.

J. Fiotto-Kaufman, A. R. Loftus, E. Todd, J. Brinkmann, K. Pal, D. Troitskii, M. Ripa, A. Belfki, C. Rager, C. Juang, et al. NNsight and NDIF: Democratizing access to open-weight foundation model internals. In *International Conference on Learning Representations*, 2025.

N. Kokhlikyan, V. Miglani, M. Martin, E. Wang, B. Alsallakh, J. Reynolds, A. Melnikov, N. Kliushkina, C. Araya, S. Yan, and O. Reblitz-Richardson. Captum: A unified and generic model interpretability library for PyTorch. *arXiv preprint arXiv:2009.07896*, 2020.

A. Mishra. Dissecting Chronos: Sparse autoencoders reveal causal feature hierarchies in time series foundation models. In *ICLR 2026 Workshop on Time Series in the Age of Large Models*, 2026. arXiv:2603.10071.

N. Nanda and J. Bloom. TransformerLens. `https://github.com/TransformerLensOrg/TransformerLens`, 2022.

K. Oublal, Q. Bouniot, Q. Gan, S. Clémençon, and Z. Akata. TimeSAE: Causal sparse decoding for faithful explanations of black-box time series models. *arXiv preprint arXiv:2601.09776*, 2026.

Z. Wu, A. Geiger, A. Arora, J. Huang, Z. Wang, N. D. Goodman, C. D. Manning, and C. Potts. pyvene: A library for understanding and improving PyTorch models via interventions. In *Proceedings of NAACL-HLT (System Demonstrations)*, 2024.